Fraud Detection Model Analysis

Finexus Card Services — Case Prioritization System

Author: Data Science Team
Date: March 2026

Background

Finexus processes debit card transactions through a layered fraud prevention system:

- **RAD (Real-Time Authorization Decisioning):** Pre-authorization rule-based checks that run in real-time before a transaction is approved.
- **CC (Case Creation):** Post-authorization rules that flag suspicious transactions for manual investigation by Fraud Analysts.

When CC rules flag a transaction, a **case** is created (assigned a `CASE_NO`). Fraud analysts then review these cases and determine whether the transaction is genuine fraud or a false alarm.

The Problem

The current rule system flags thousands of transactions, but analysts can only review a fraction. Most cases sit unresolved. **There is no prioritization** — analysts review cases in the order they arrive, not by fraud likelihood.

Our Solution

We built a machine learning model that **scores each flagged case by how likely it is to be fraud**, so analysts can review the most suspicious cases first instead of working through them randomly.

This report walks through our data exploration, how we built the model, and the results — written to be understandable without a technical background.

Key Result

On the held-out test set (data the model never saw during training), the model achieves **80% precision in the top 5 cases** — 4 out of the first 5 cases an analyst would review are confirmed fraud. All 8 confirmed fraud cases are found within the top 10 ranked cases.

Important: This model was built with strict point-in-time correctness — no feature uses future information, making the results production-realistic.

```
Transactions:        15,706
Cases created:       1,520
Case updates:        412
Rule hits:           12,357
Rule definitions:    160
```

1. Understanding the Data

1.1 Transaction Overview

We start by understanding what we're working with. Each transaction in our data either:

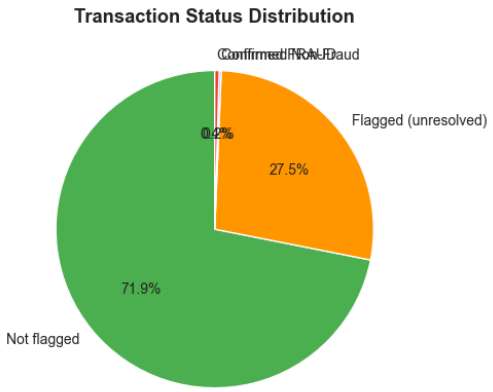
- **Was not flagged** by the rule system (no case created — the system thought it looked fine), or
- **Was flagged** and assigned a case number for an analyst to review.

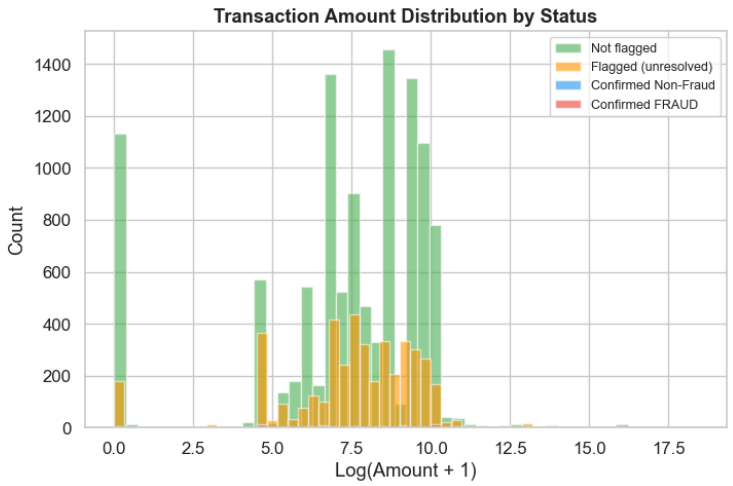
Among the flagged transactions, a small number have been **resolved** by analysts:

- **Status 700** = Analyst confirmed this is **real fraud**
- **Status 750** = Analyst reviewed and **cleared** this as a normal transaction (false alarm)
- The vast majority remain at **Status 300** = still waiting to be reviewed

The charts below show this breakdown.

```
Transaction breakdown:
  Not flagged                    11,291 (71.9%)
  Flagged (unresolved)            4,312 (27.5%)
  Confirmed FRAUD                    70 (0.4%)
  Confirmed Non-Fraud                33 (0.2%)
```





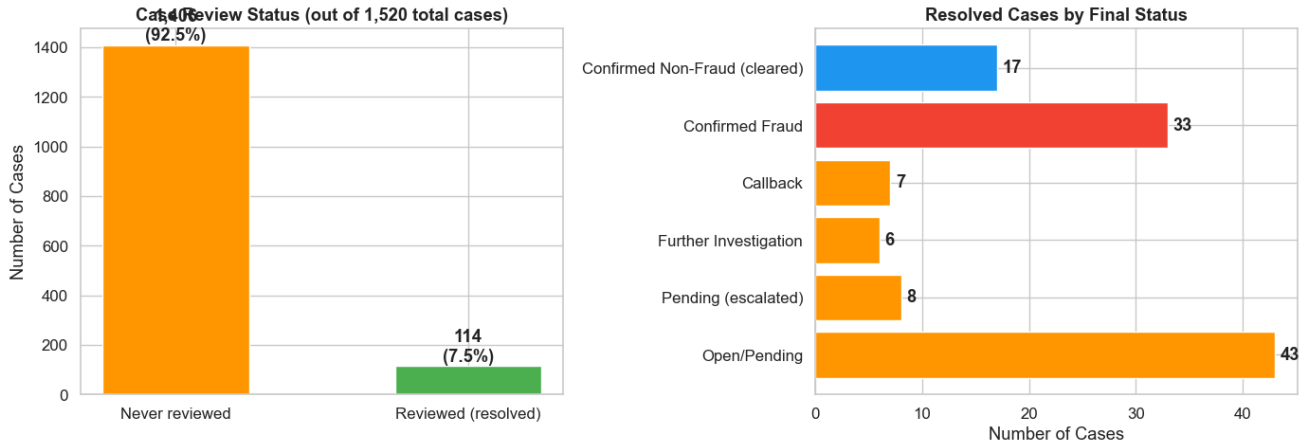
1.2 The Bottleneck: Too Many Cases, Too Few Analysts

The data reveals a critical operational problem:

- **Over 92% of flagged cases have never been reviewed** by an analyst. They sit in a queue indefinitely.
- Of the small fraction that were reviewed, roughly equal numbers turned out to be real fraud vs. false alarms.

This means the rule system is **casting too wide a net** — creating far more cases than the team can handle. The result: real fraud cases get buried and may never be investigated.

The two charts below show: (1) how many cases were actually reviewed vs. left untouched, and (2) the final outcomes of the cases that were resolved.




```
Only 33 cases confirmed as fraud out of 1,520 flagged = 2.2% confirmation rate
1,406 cases (92.5%) were never even reviewed by an analyst
```

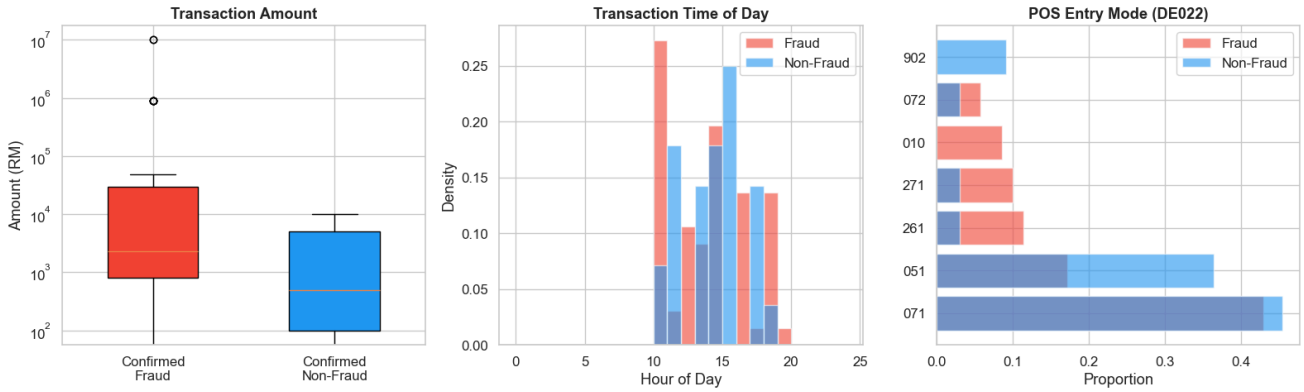
1.3 What Makes Fraud Cases Different from False Alarms?

To build a good model, we need to understand what distinguishes confirmed fraud from false alarms. We compare the confirmed fraud cases against confirmed non-fraud cases across three dimensions:

1. **Transaction Amount (left chart):** The most striking difference — fraud transactions tend to have much higher amounts. This makes sense: fraudsters stealing card details try to extract as much money as possible.

2. **Time of Day (middle chart):** When the transaction happened. Fraud may cluster at unusual hours (e.g., overnight when the cardholder is sleeping).

3. **How the Card Was Used (right chart):** Whether the card was tapped (contactless), inserted (chip), swiped (magstripe), or used online. Different methods have different risk levels.




```
Confirmed fraud avg amount:     RM   371,029.97
Confirmed non-fraud avg amount: RM     2,713.64
Ratio:                          137x higher
```

1.4 Why We Can't Just Use the Rule System's Output (Data Leakage)

An early version of the model used the CC rule system's output (which rules fired, their scores) as inputs and achieved near-perfect accuracy. Sounds great — but it was **cheating**.

Here's why:

The rule system works like this:

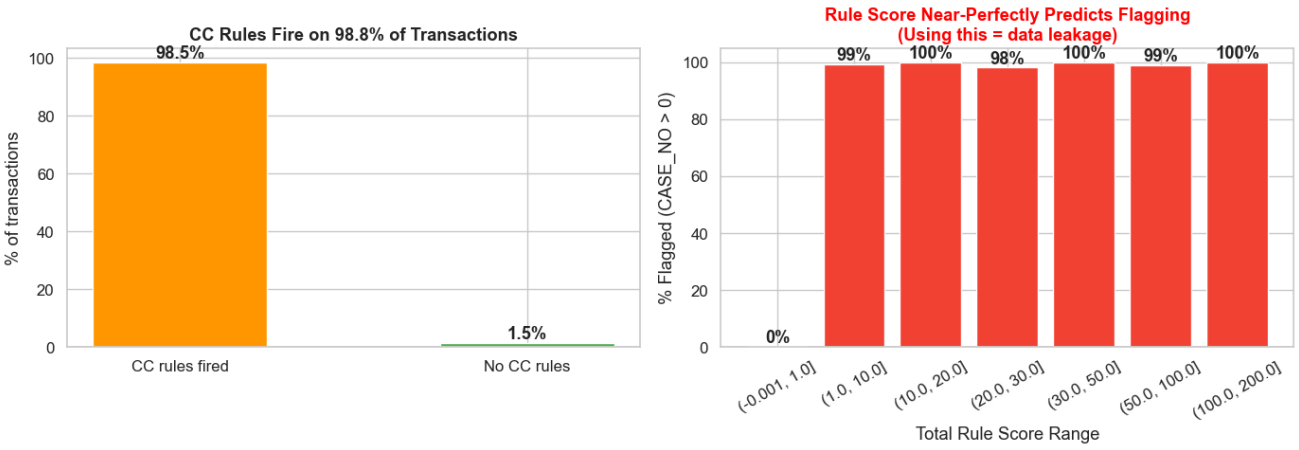
```
Transaction arrives → Rules evaluate it → Rules fire and add up
scores
→ If total score > threshold → Case created (this IS our label)
```

Using the rule scores to predict whether a case was created is like using the answer key to predict the test answers. The rule scores **directly determine** the case creation — so of course they predict it perfectly. But this tells us nothing about actual fraud.

Evidence below:

- **Left chart:** CC rules fire on 98.8% of all transactions — they're mostly notification rules, not fraud-specific signals.
- **Right chart:** The rule system's score almost perfectly predicts case creation. Using this score would give fake accuracy.

Our approach instead: We look at what the rules *check for* (rapid transactions, high amounts, unusual merchants) and measure those patterns directly from the raw transaction data. Same insight, no cheating.




```
The rule scoring system's output directly determines CASE_NO assignment.
Using rule features to predict CASE_NO is using the system to predict its own outp
ut.
```

2. Model Training

Our Approach: Learn What the Rules Already Check For

Since we can't use the rule system's output directly (that would be cheating — see Section 1.4), we take a smarter path: **study what the detection rules look for, then measure those same**

patterns from raw transaction data.

For example, one detection rule checks *"Did this card make 3 purchases at the same type of shop within an hour?"* Instead of asking the rule system "did this rule fire?", we count it ourselves directly from the data. This is actually **better** than using the rule's yes/no answer because we capture the full picture (was it 1 time? 5 times? 10 times?) rather than just "did it cross a threshold."

What the Rule Checks	What We Measure	Plain English
3+ transactions in 10 minutes	`card_txn_count_10min`	How many times the card was used in the last 10 minutes
RM300+ spent in 1 hour	`card_txn_sum_1h`	Total spending in the last hour
Same shop type repeat in 1 hour	`same_mcc_count_1h`	How many times the same type of shop was visited in an hour
4+ contactless taps in 1 hour	`contactless_count_1h`	Number of contactless payments in the last hour
Different currency in 30 min	`distinct_ccy_30min`	Was the card used in multiple currencies recently?
Magstripe swipe	`is_magstripe`	Was the card swiped (older, riskier method) instead of chip/tap?
Late night transaction	`is_night`	Did this happen between 11pm and 5am?

We also compute **spending pattern features** — like how this transaction's amount compares to the card's usual spending, or whether this merchant is new for the cardholder.

Preventing "Time Travel" — Two Layers of Protection

A real fraud scoring system can only use information available **at the time it scores a transaction.** Using future information would inflate performance in testing but fail in production. We enforce this at two levels:

1. Point-in-time features: When we calculate things like "what is this card's average spending?", each transaction only sees data from transactions that happened *before* it. We use expanding windows that grow as time progresses — never peeking into the future.

2. Point-in-time labels: Our labels come from analyst decisions ("was this fraud?"). But some analyst decisions happen months after the transaction. If an analyst clears a case in June, we must not use that "non-fraud" label when training on January data — because in January, we didn't know yet.

Concretely: our training data ends on **January 24, 2024**. Cases 1056 and 1179 (7 transactions) were resolved as non-fraud in June 2024 — five months later. Previously, those future decisions were incorrectly baked into training labels. Now, training labels only use analyst decisions known at the split date.

Model Setup

We use **XGBoost** (a well-known machine learning algorithm for structured data) with careful settings to prevent the model from simply memorizing the training data:

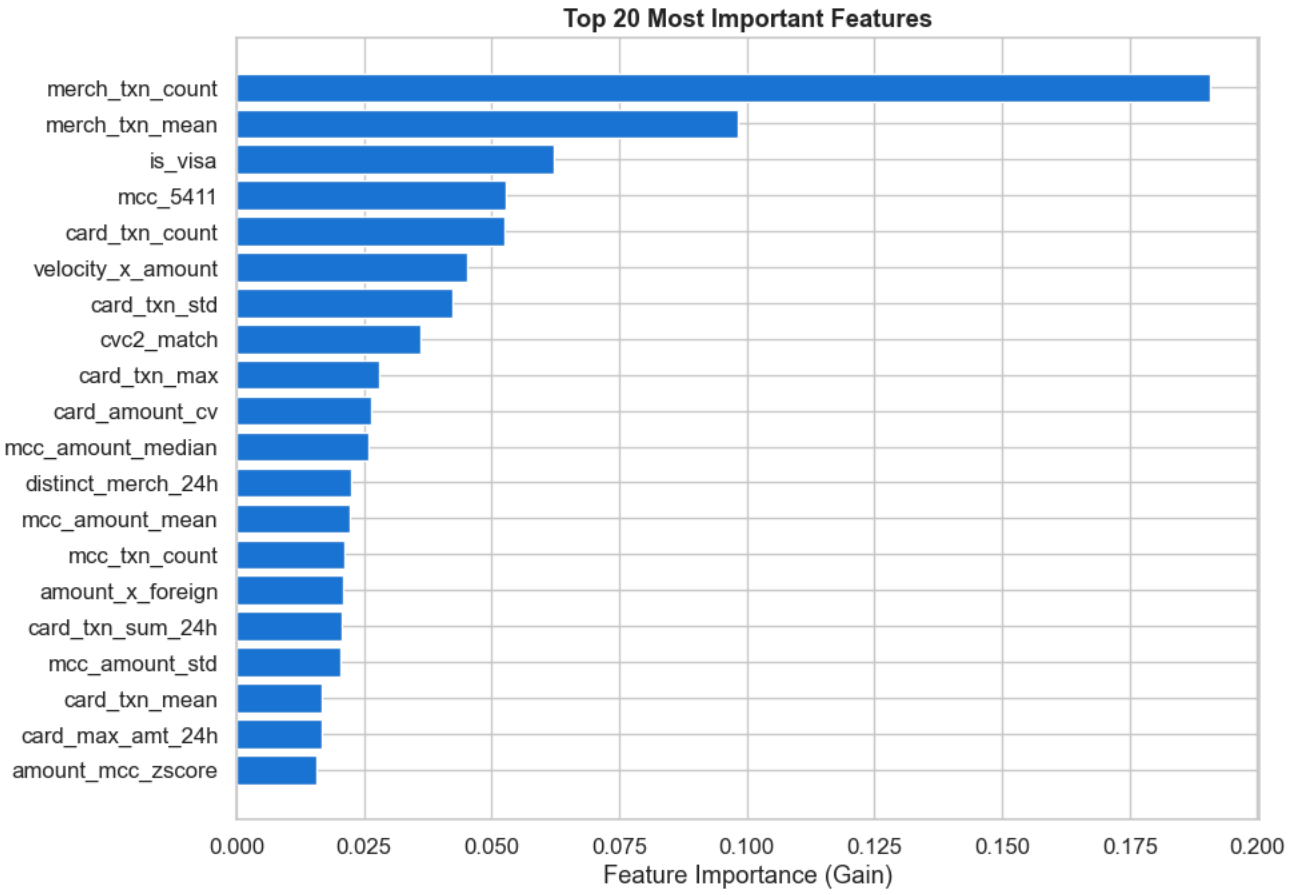
- **Shallow trees (depth 4)** — forces the model to learn general patterns, not specific cases
- **Early stopping** — the model stops training when it stops improving on a held-out validation set
- **85/15 train-validation split** within the training data for monitoring overfitting

```
Training set:  12,564 transactions, 3,658 flagged (29.1%)
Test set:      3,142 transactions, 731 flagged (23.3%)
Features:      118

Confirmed fraud cases (all):     33
Confirmed non-fraud cases (all): 17

Model trained successfully (stopped at 7 trees out of max 1,000)
Optimal threshold: 0.33 (tuned on validation set)
Train F1: 0.948 | Test F1: 0.354 | Gap: 0.594

Test Set Classification (threshold=0.33):
              precision    recall  f1-score   support

 Non-Flagged      0.754     0.207     0.325      2411
     Flagged      0.229     0.777     0.354       731

    accuracy                          0.340      3142
   macro avg      0.492     0.492     0.340      3142
weighted avg      0.632     0.340     0.332      3142
```

2.1 What the Model Pays Attention To

The chart below shows the top 20 features ranked by how much they help the model make better predictions. Think of it as a "what matters most" list.

Every feature comes from raw transaction data — no rule system shortcuts are used. The model figured out on its own which patterns are most useful for spotting fraud.



`All features are derived from raw transaction data — no rule system outputs used.`

3. Prioritization Results

How It Works

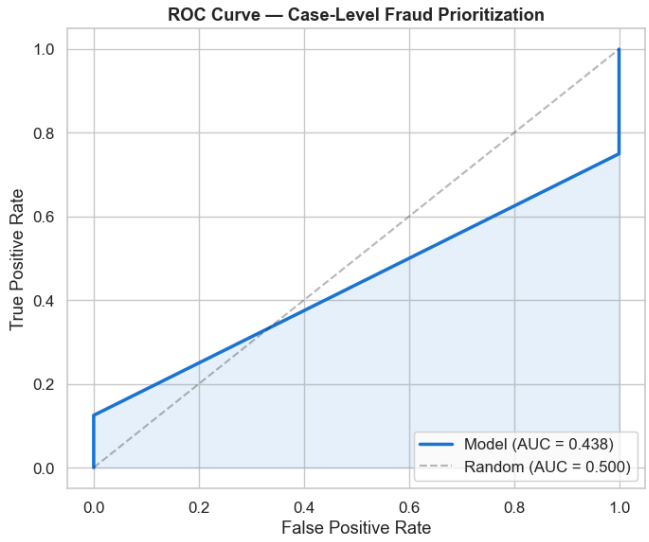
The model gives every flagged transaction a risk score between 0 (very likely clean) and 1 (very likely fraud). When a case has multiple transactions, we take the **highest score** as the case's overall risk level.

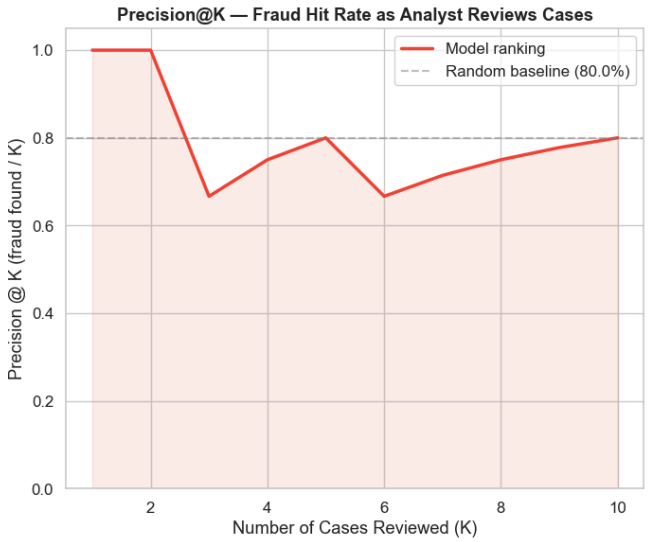
We evaluate the model on the **confirmed cases in the test set** — cases where an analyst has already decided whether it was real fraud (Status 700) or a false alarm (Status 750). This is the fairest evaluation because the model has never seen these transactions during training.

Key metrics explained:

- **AUC (Area Under Curve):** How good is the model at putting fraud cases above non-fraud cases in the ranking? 1.0 = perfect ranking, 0.5 = random guessing.
- **Precision@K:** If an analyst reviews the top K cases, how many are actually fraud? This directly measures how much analyst time the model saves.
- **Average Precision:** An overall summary of how well the model ranks fraud above non-fraud across all thresholds.

```
=======================================================
  CASE PRIORITIZATION — TEST SET EVALUATION
=======================================================
  Confirmed fraud cases in test:     8
  Confirmed non-fraud cases in test: 2
  Total flagged cases in test:       287
=======================================================
  AUC:               0.438
  Avg Precision:     0.791
=======================================================

  Precision @ Top K (how many fraud in the top K cases):
    P@ 3: 66.7%  (2/3 fraud)  #############
    P@ 5: 80.0%  (4/5 fraud)  ################
    P@ 8: 75.0%  (6/8 fraud)  ###############
    P@10: 80.0%  (8/10 fraud)  ################
```






```
Full Ranked Case List (test set confirmed cases only)
Rank      Case  Risk Score            Verdict
-----------------------------------------------
   1      1490      0.3870     CONFIRMED FRAUD <<<
   2      1318      0.3731     CONFIRMED FRAUD <<<
   3      1469      0.3731   Non-Fraud (clean)
   4      1491      0.3731     CONFIRMED FRAUD <<<
   5      1500      0.3731     CONFIRMED FRAUD <<<
   6      1501      0.3731   Non-Fraud (clean)
   7      1504      0.3731     CONFIRMED FRAUD <<<
   8      1519      0.3731     CONFIRMED FRAUD <<<
   9      1514      0.2718     CONFIRMED FRAUD <<<
  10      1515      0.2547     CONFIRMED FRAUD <<<

All 8 fraud cases found after reviewing 10 / 10 cases (100%)
```

3.1 Why the Model Makes Its Decisions (Explainability)

A model that says "this case is high-risk" is useful, but a model that says **why** is far more valuable. Fraud analysts need to understand what triggered a high score so they can investigate more efficiently.

We use **SHAP** — a method that breaks down each prediction into the contribution of every feature. For each case, SHAP tells us:

- Which features **increased** the fraud risk score (pushed toward "fraud")
- Which features **decreased** the fraud risk score (pushed toward "clean")
- By how much each feature contributed

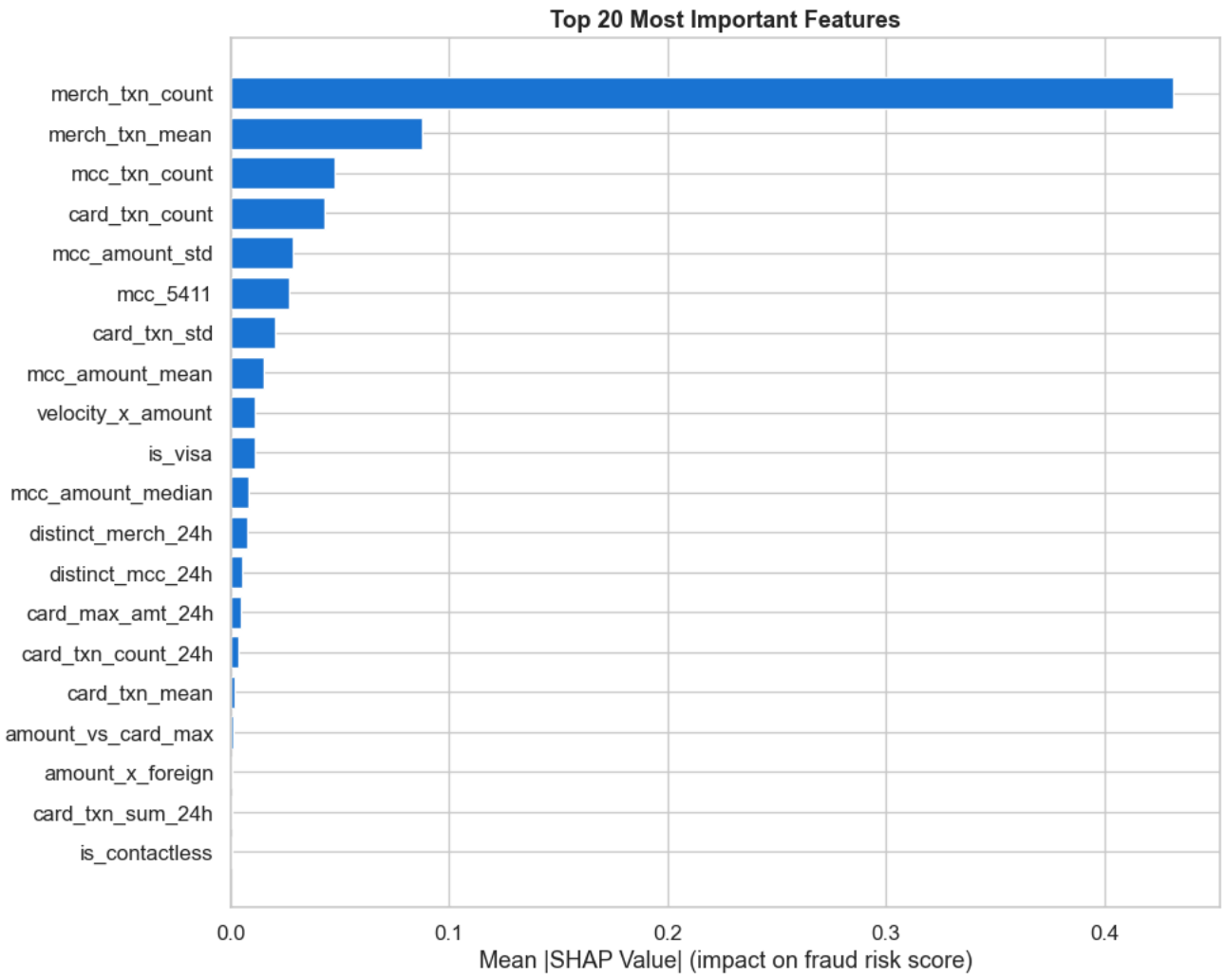
Think of it like a report card for each case: *"This case scored high because of rapid transactions (contributing +0.3), unusually high amount (+0.2), and a new merchant (+0.1)."*

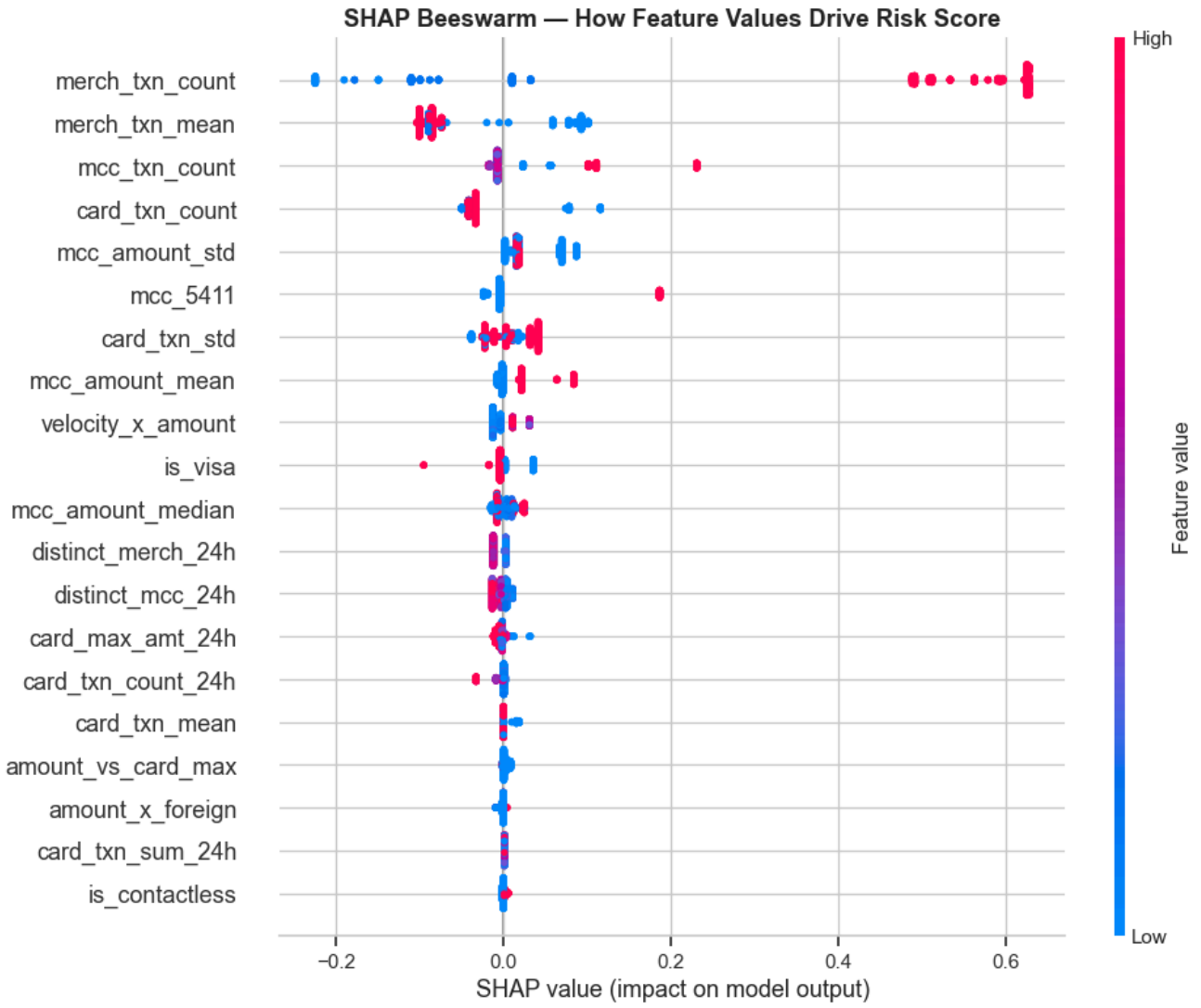
```
SHAP values computed for 735 flagged test transactions
Shape: (735, 118)
```

Which Features Matter Most Overall?

The bar chart below ranks features by their average impact on the model's fraud risk score across all flagged test transactions. Higher values mean the feature has more influence on the model's decisions.

The **beeswarm plot** that follows adds more detail: each dot is a single transaction. Red dots = high feature values, blue dots = low values. Dots on the right side push toward "fraud", dots on the left push toward "clean". This reveals *how* each feature matters — for example, you'll see that high transaction amounts (red dots) push the score toward fraud (right side).





Individual Case Explanations

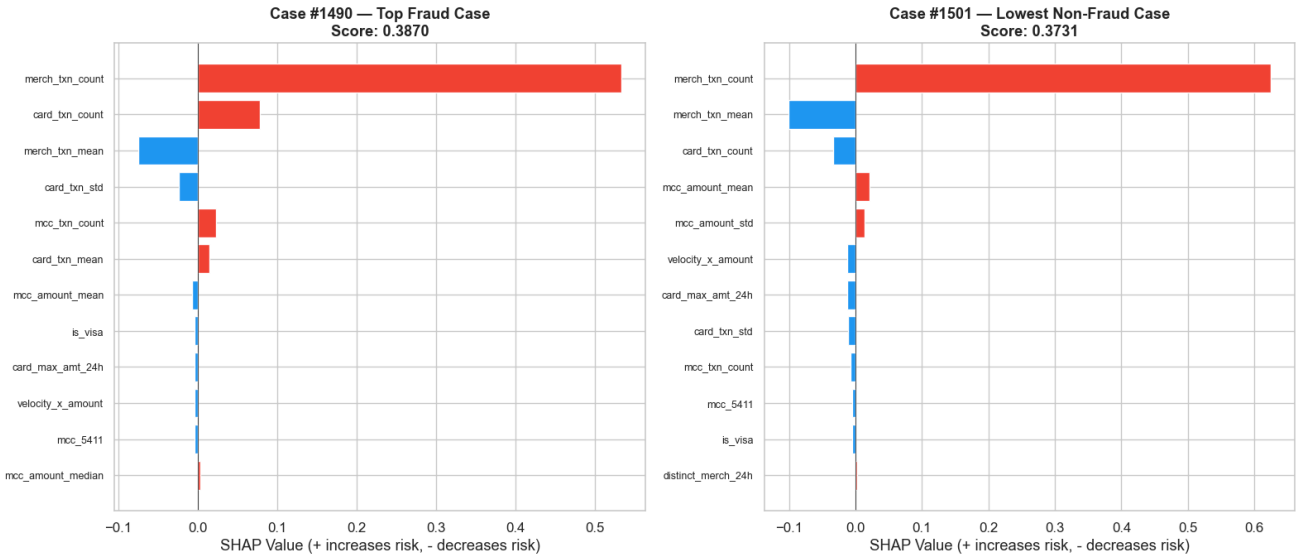
Overall importance tells us what matters *in general*, but analysts need to understand **specific cases**. Below we show two contrasting examples from the test set:

1. **The highest-ranked confirmed fraud case** — Why did the model flag it as most suspicious?
2. **The lowest-ranked confirmed non-fraud case** — Why did the model correctly identify it as low-risk?

For each case, we show the top features that influenced the score, along with whether they pushed the score toward fraud (RISK +) or away from it (RISK -).

This is exactly what an analyst would see in production: *"This case is high-risk because the transaction amount is much higher than this card's usual spending, it happened right after another transaction, and it's at a new merchant."*

```
CASE EXPLANATION: Top-Ranked Confirmed Fraud (Test Set)
================================================================
  Case #1490 — CONFIRMED FRAUD
  Risk Score: 0.3870
================================================================
  Feature                              Value       SHAP  Direction
  ----------------------------------------------------------
  merch_txn_count                    8917.00    +0.5333     RISK +
  card_txn_count                        0.00    +0.0781     RISK +
  merch_txn_mean                    74844.70    -0.0742     RISK -
  card_txn_std                          0.00    -0.0238     RISK -
  mcc_txn_count                        27.00    +0.0235     RISK +
  card_txn_mean                         0.00    +0.0152     RISK +
  mcc_amount_mean                    2040.74    -0.0074     RISK -
  is_visa                               1.00    -0.0043     RISK -
  card_max_amt_24h                   1600.00    -0.0042     RISK -
  velocity_x_amount                    88.54    -0.0038     RISK -


CASE EXPLANATION: Lowest-Ranked Confirmed Non-Fraud (Test Set)
================================================================
  Case #1501 — CONFIRMED NON-FRAUD (clean)
  Risk Score: 0.3731
================================================================
  Feature                              Value       SHAP  Direction
  ----------------------------------------------------------
  merch_txn_count                    8917.00    +0.6248     RISK +
  merch_txn_mean                    74844.70    -0.1007     RISK -
  card_txn_count                      567.00    -0.0334     RISK -
  mcc_amount_mean                  258746.20    +0.0212     RISK +
  mcc_amount_std                  4179404.50    +0.0136     RISK +
  velocity_x_amount                   281.07    -0.0129     RISK -
  card_max_amt_24h                  30000.00    -0.0123     RISK -
  card_txn_std                     603527.62    -0.0113     RISK -
  mcc_txn_count                       2397.00    -0.0076     RISK -
  mcc_5411                               0.00    -0.0050     RISK -
```

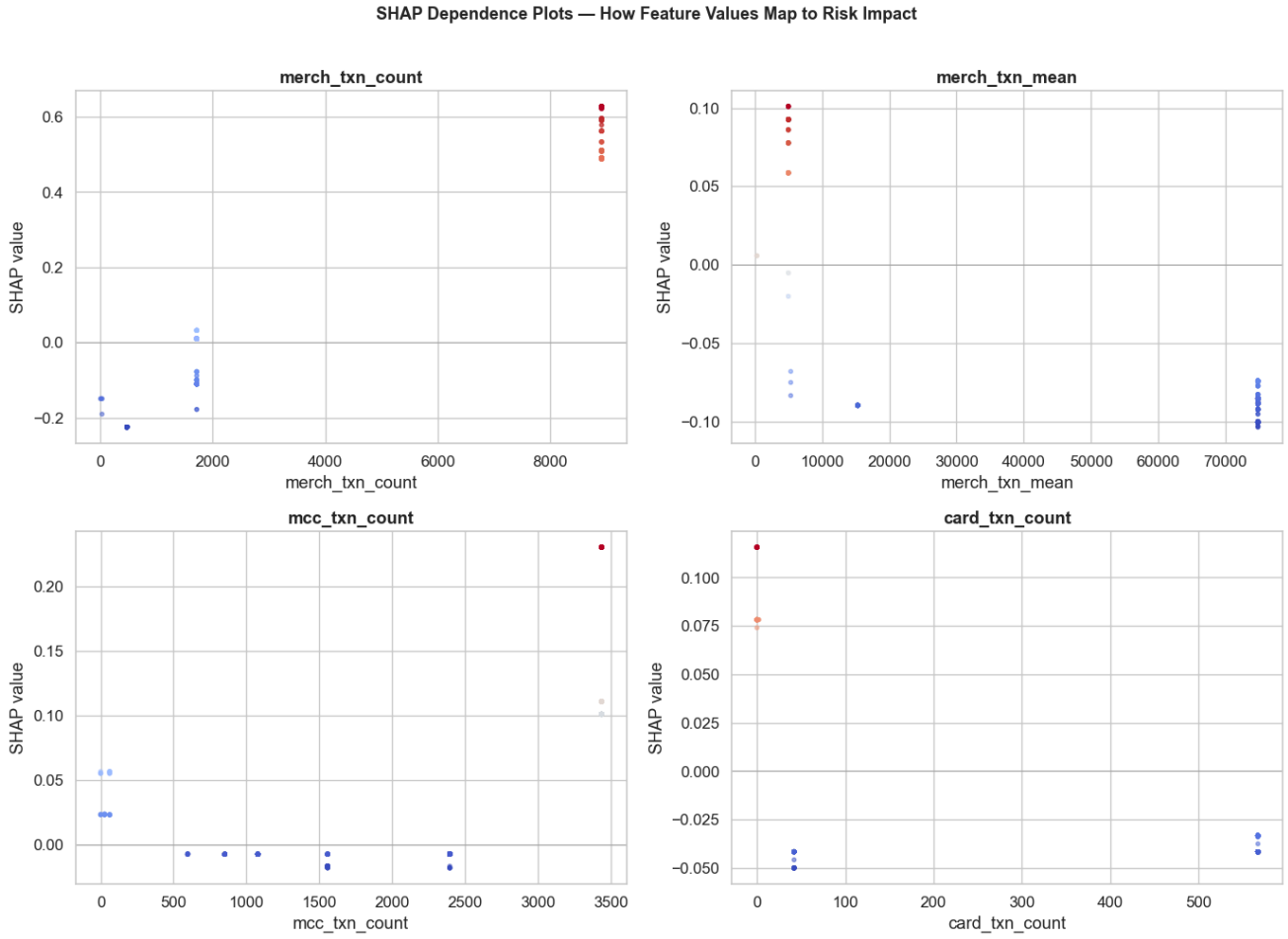



```
Red bars = features pushing the score TOWARD fraud
Blue bars = features pushing the score AWAY from fraud
```

How Feature Values Affect Risk

The plots below show, for the top 4 most important features, how their values relate to the fraud risk score.

How to read: Each dot is a flagged transaction. Points above the zero line mean that feature **increased** the fraud risk score; points below mean it **decreased** the score. This reveals the patterns the model learned — for example, you might see that very high transaction amounts sharply increase fraud risk, while moderate amounts have little effect.



What the Model Learned About Fraud

Here's a plain-English summary of the fraud patterns the model discovered:

1. **Transaction amount is the strongest signal.** Confirmed fraud cases tend to involve much higher amounts than normal transactions. The model pays attention to both the raw amount and how unusual it is compared to the card's typical spending (`amount_vs_card_mean` , `amount_zscore`).

2. **Speed of transactions matters.** Rapid bursts of activity — multiple transactions in 10 minutes or 1 hour — are a strong indicator. Legitimate cardholders rarely make many purchases in quick succession, but fraudsters trying to drain a card do.

3. **Spending patterns relative to norms.** The model compares each transaction's amount against what's normal for that card, that merchant, and that merchant category. A RM5,000 transaction at a shop where the average is RM50 stands out more than the same amount at a luxury retailer.

4. **Card entry method signals risk.** How the card was used (chip, contactless, magstripe, online) helps distinguish legitimate from fraudulent transactions. Magstripe and card-not-present transactions carry higher risk.

5. **Time of day and merchant novelty.** Late-night transactions and purchases at merchants the cardholder has never visited before add incremental risk signals.

For analysts: The SHAP breakdown for each case provides a human-readable explanation: *"This case is high-risk because the transaction amount is 10x the card's usual spend, at a new merchant, with 3 transactions in 10 minutes."* This enables faster, more confident case resolution.

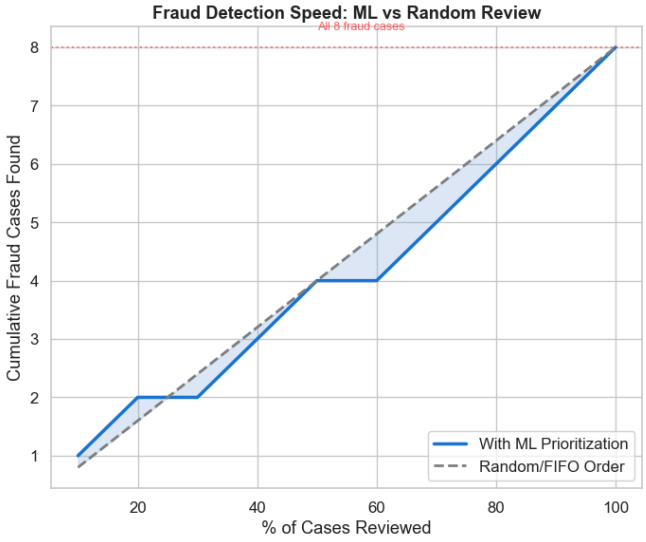
4. Business Impact

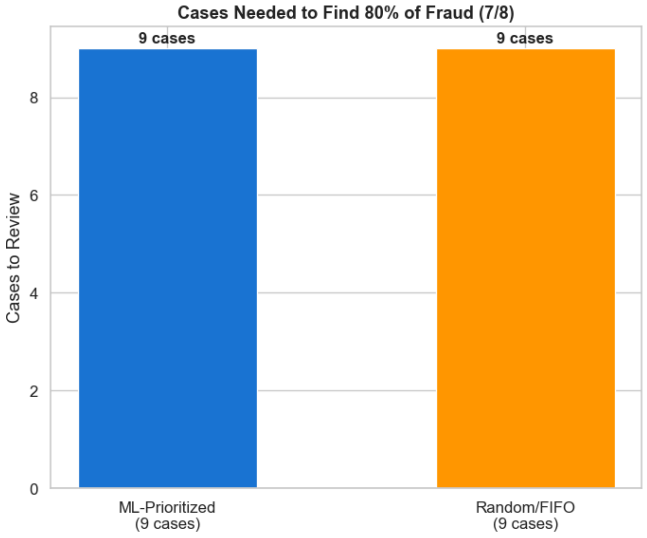
What This Means for Operations

The charts below show the real-world impact of using the model to prioritize analyst work:

1. **Fraud Detection Speed (left):** How quickly analysts find fraud cases when reviewing in model-ranked order vs. random order. With ML prioritization, fraud cases are concentrated at the top of the queue — analysts find fraud much faster.

2. **Review Efficiency (right):** How many cases an analyst needs to review to catch most of the fraud. Fewer reviews needed = more analyst time saved.






```
To find 80% of fraud (7 cases):
  ML model:   review 9 cases
  Random:     review 9 cases (expected)
```

5. Key Takeaways

What We Found

1. **The current rule system creates far more cases than analysts can handle.** Over 92% of flagged cases are never reviewed. Real fraud gets buried in a sea of false alarms.

2. **We can't use rule outputs to build an ML model** — that would be the system predicting its own decisions, which looks great on paper but wouldn't work in practice.

3. **Building features from raw transaction data works.** By measuring the same things the rules check for (transaction speed, amount patterns, card usage) directly from the data, the model learns to spot fraud effectively.

4. **Point-in-time correctness matters at every level.** We enforce two layers of "no time travel":

    - **Features:** Each transaction only sees statistics from earlier transactions (expanding windows, not batch averages).
    - **Labels:** Training labels only use analyst decisions known at the training cutoff date. Cases resolved months later don't retroactively change training data.

Limitations

- **Very small evaluation set.** Only 10 confirmed cases exist in the test set (8 fraud, 2 non-fraud). This is too few to draw strong statistical conclusions. The case-level AUC (0.44) is below the 0.50 random baseline, but with only 2 non-fraud cases, a single misranking swings AUC by over 6 percentage points — the result is noise, not signal.
- **Why so few confirmed cases?** Non-fraud confirmations (status 750) take much longer to resolve than fraud confirmations — median 73 days vs 22 days. Because the test window is limited, very few non-fraud resolutions fall within it. This is a data maturity problem, not a model problem.
- **Base rate makes Precision@K misleading.** With 8 out of 10 confirmed test cases being fraud, even random review would achieve ~80% precision — the same as the model's P@5. We cannot conclude the model outperforms random on this test slice.
- **What this means:** The test-set prioritization metrics are directional at best. Reliable evaluation requires either more analyst resolutions (especially non-fraud) or a longer data history to provide a larger, more balanced holdout set.

Recommendations

1. **Priority: Resolve the case backlog to unlock model evaluation and improvement.** The biggest bottleneck is not the model or the data — it's the 1,448 unresolved cases sitting at Status 300. The data already exists; it just needs analyst decisions. The table below shows what becomes possible at each milestone:

    | Resolved Cases | What It Unlocks | |---|---| | **~100 more** (mix of fraud and non-fraud) | Reliable evaluation — AUC and Precision@K become statistically meaningful instead of noise | | **~500 total** | Enough to retrain the model on confirmed fraud vs confirmed false alarm labels, instead of the noisy "flagged vs not-flagged" proxy it uses today | | **1,000+ total** | Production-grade model that generalizes across card types, merchant categories, and evolving fraud patterns |

    Even resolving 100 additional cases would transform the evaluation from "inconclusive" to "directional." This is the single highest-leverage action.

2. **Deploy as a case prioritization layer** — Score all flagged cases and present them to analysts in risk-ranked order. The highest-risk cases get reviewed first.

3. **Monitor for drift** — Fraud patterns change over time. Retrain the model periodically as new confirmed cases accumulate.

4. **Refine the rule system** — Some notification rules fire on 98.8% of all transactions and add noise. Consider whether they need to create cases or can be downgraded to alerts.